

Academic Manager and Analysis

Mini-Project for: Undergraduate Python Course Developed using PythonAcademic Year: 2025–26

Meet the Innovators Behind the Project

This project was a collaborative effort by a dedicated team of students, guided by our esteemed faculty.

Team Member 1

Sanjana Pandharkar - 1262251827

Team Member 2

Pragnya Pal- 1262252574

Team Member 3

Tania Roy-1262251952

Team Member 4

Vardhini Chavan-1262251614

Team Member 5

Samiksha satav-1262253498

Faculty Incharge

Deepali Mam

The Challenge: Manual Academic Management

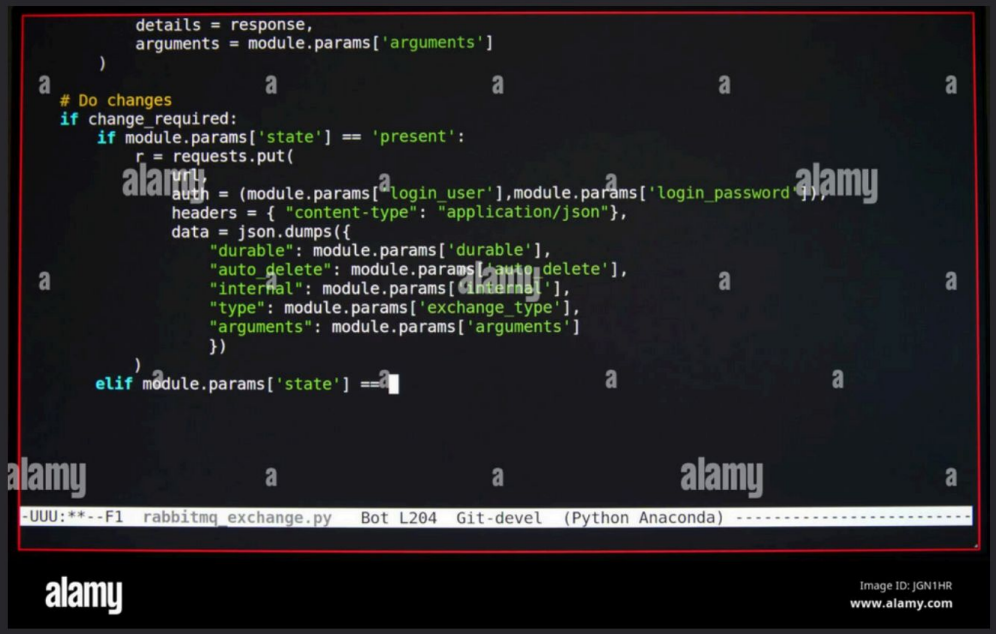
Problem Statement

Manual academic assessment management is prone to errors and is time-consuming for both educators and students.



- Teachers and coordinators struggle to maintain accurate records of tests across multiple subjects like Maths, Science, English, History, and Computer.
- Tracking assignment submission status is often a cumbersome task.
- Students lack a systematic way to monitor their performance and submission records efficiently.

Our Solution: Academic Record Manager



Project Objective

To develop a simple, yet robust, Python-based academic record manager that stores and displays each student's details in a tabular format, ensuring efficient and accurate continuous assessment data management.

What the Project Does

- Stores comprehensive student information in a structured table.
- Includes dedicated columns for test marks in Maths, Science, English, History, and Computer.
- Manages assignment status with "Assignment Completed" and "Assignment Due" columns (randomized between 1-5 to simulate dynamic data).
- Allows operations such as adding new student data, updating marks, checking assignment submissions, and displaying the complete table.

Crafting the Dataset: The Foundation

The dataset forms the backbone of our academic manager, meticulously designed to track student performance.

Dataset Description

- Captures comprehensive student academic records.
- Includes Roll Number, Student Name, Test marks for various subjects, and Assignment Completed/Due status.
- Data can be hard-coded, loaded from a CSV, or interactively entered by the user.

Example Data Structure

Roll No.	Name	Test Marks	Ass. Comp.	Ass. Due
101	Alice	85 (M), 90 (S)	4	1
102	Bob	78 (M), 82 (S)	3	2

This structured format ensures the data is both easy to process and highly suitable for academic performance tracking within a mini-project scope.

The Engine: Project Algorithm

Our Python script follows a clear, step-by-step logic to manage student records effectively.

01

Start & Data Initialization

Begin execution; create or load the dataset (list/dictionary/CSV).

02

Define Core Functions

Establish functions for adding records, managing assignment status, and displaying data.

03

User Interaction

Accept user input to determine desired actions.

04

Conditional Execution

Use conditional statements (if/else) to execute the appropriate function based on user choice.

05

Perform Operations

Execute data manipulation on the dataset.

06

Display Results

Present updated information in a clear table format.

07

Loop or Exit

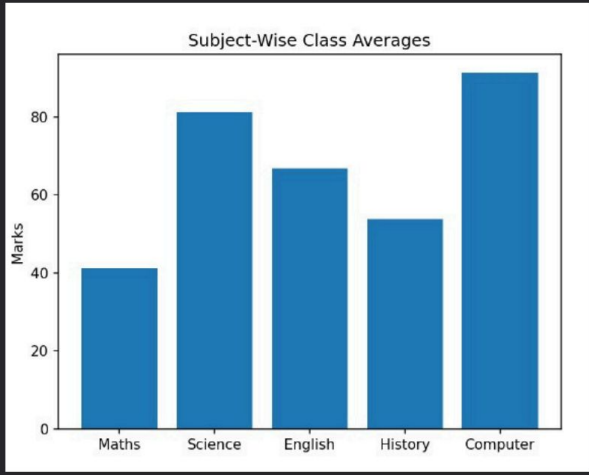
Repeat the process until the user chooses to exit, then end.

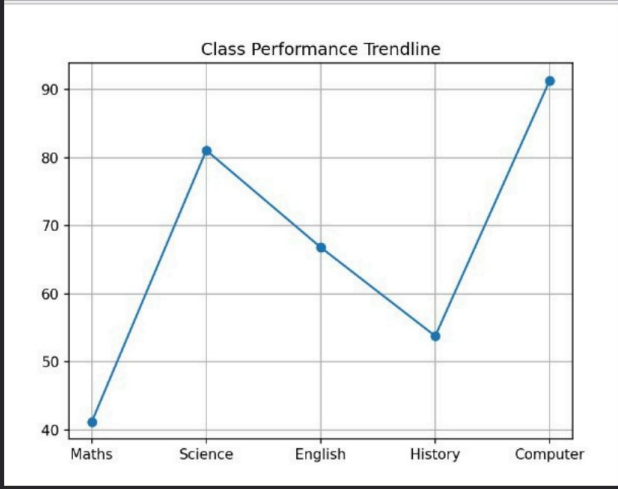
Navigating the Process: Flowchart

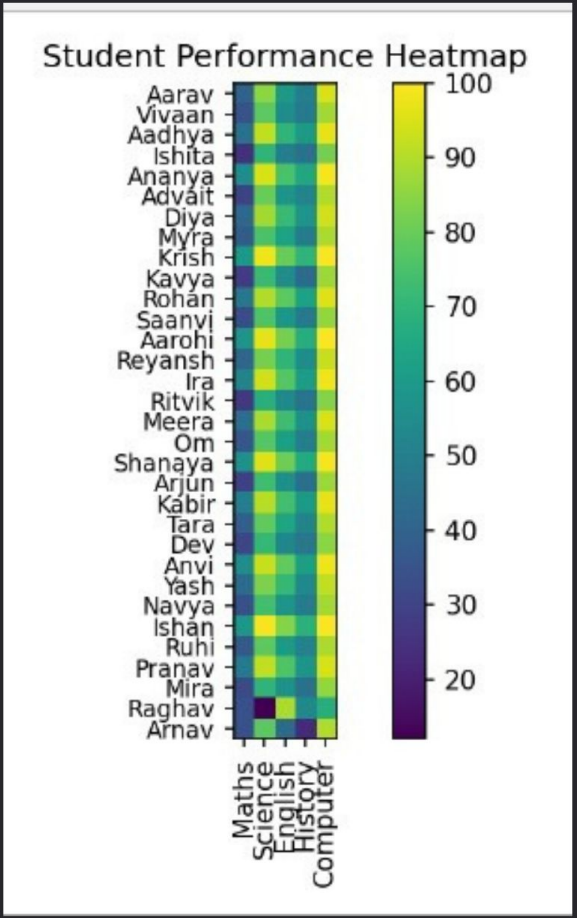
This flowchart visually outlines the logical flow of our academic record manager, from startup to exit.

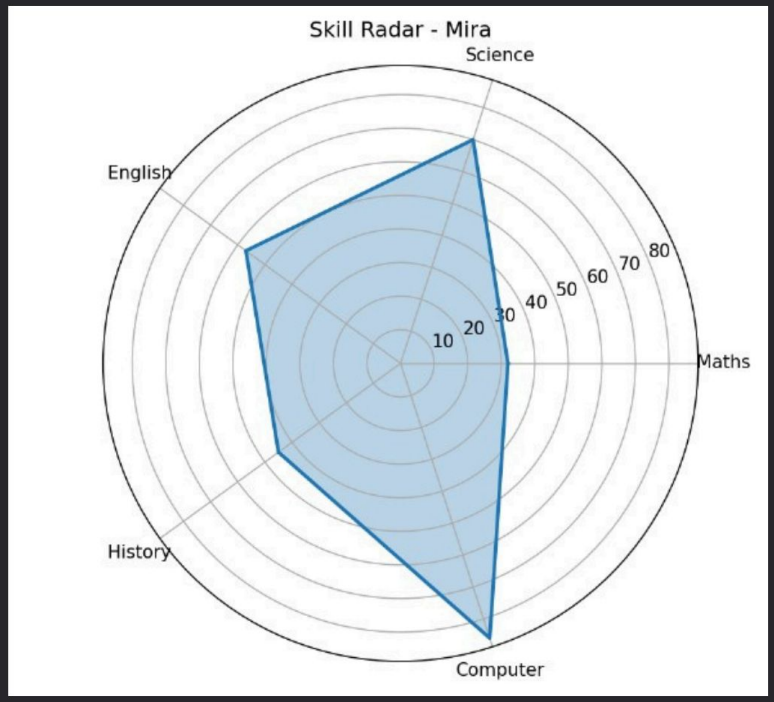


In Action: Screenshots of Results









Key Takeaways: Project Conclusion

Project Success

The project successfully manages student academic records using Python, significantly reducing manual effort and improving data accuracy.

Core Demonstrations

It demonstrates practical application of file handling, data structures, functions, modular coding, and user interaction within Python.

Improved Clarity

Provides a structured tabular view for easy comprehension of student performance and assessment data.

Future Enhancements & Potential

Our Academic/Test Schedule Manager can be significantly expanded to offer even greater functionality and user experience.

GUI Development

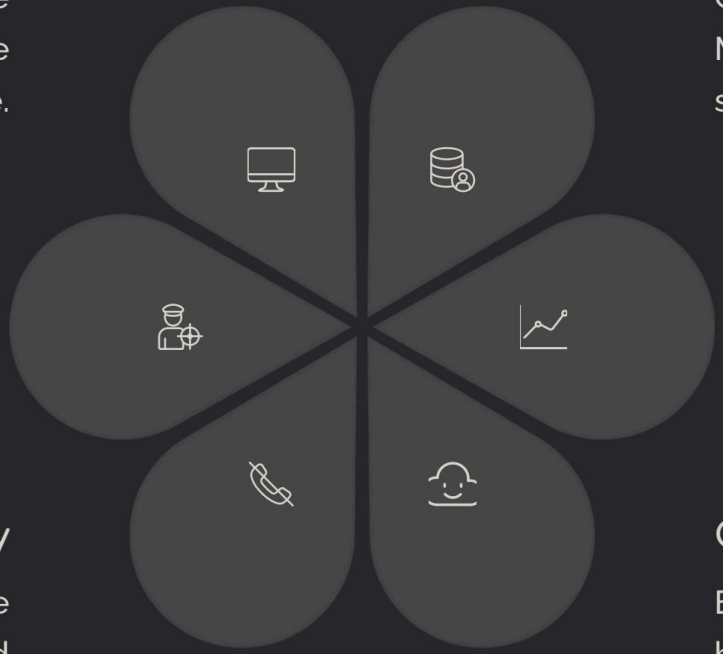
Implement a Graphical User Interface using libraries like Tkinter for a more user-friendly experience.

Collaborative Features

Add functionalities for multi-user access, roles, and real-time collaboration among educators.

Mobile Accessibility

Create a mobile application or responsive web interface for on-the-go access and updates.



Database Integration

Connect to a robust database (e.g., MySQL, SQLite) for enhanced data storage, retrieval, and scalability.

Automated Reports

Develop features for automated generation of performance reports, attendance sheets, and progress summaries.

Cloud Integration

Explore cloud-based solutions for data backup, synchronization, and remote access.

Thank You!!